Année scolaire 2026-2027

Lycée Victor Hugo - Colomiers

Première NSI

RD
Représentation et traitement de données

Thomas Ricart

ricart.lvh@laposte.net

https://thomasricart.github.io

31 mars 2026

RD01 REPRÉSENTATION DES ENTIERS POSITIFS

I. Origine du binaire en informatique

Pour communiquer, deux êtres humains doivent parler la même langue. L'échange d'informations entre les composants d'un ordinateur repose sur le même principe.

Dans les années 40, lorsqu'il a fallu passer de l'idée abstraite de machine de Turing (modèle mathématique) à la réalisation concrète du premier ordinateur (machine physique), la question des composants matériels à choisir s'est posée (transistor, lampe, DEL, aimant, tube, bande, ...). Pour ces composants, il est possible de réduire leur fonctionnement à 2 états exclusifs l'un de l'autre : état 0 et état 1.

- le transistor est bloqué (état 0) ou passant (état 1),
- l'interrupteur est ouvert (état 0) ou fermé (état 1),
- la lampe est éteinte (état 0) ou allumée (état 1),
- le ruban est trouée (état 0) ou non (état 1),

Le langage de base finalement choisie pour l'informatique, et encore utilisé aujourd'hui, est le binaire dont l'unité est le **bit** : binary digit (ou nombre binaire), pouvant prendre comme seules valeurs **0 ou 1**.

Exercice 1

Combien d'états différents peut-on coder avec 1 bit ? 2 bits ? 8 bits ? n bits ? .....

II. Représentation binaire

Exercice 2

Représentation binaire d'un entier naturel
Sur 8 bits, donner la représentation des premiers entiers naturels :

V. décimale	V. binaire		V. décimale	V. binaire		V. décimale	V. binaire		V. décimale	V. binaire
0			8			16			24	
1			9			17			25	
2			10			18			26	
3			11			19			27	
4			12			20			28	
5			13			21			29	
6			14			22			30	
7			15			23			31	

Propriété 1 : A retenir

- un entier naturel est représenté en binaire par une suite de bits (0 et 1).
- un entier naturel codé sur n bits peut prendre 2^n valeurs différentes, de 0 à $2^n - 1$.
- avec 8 bits, on peut représenter les entiers naturels de 0 à 255.
- le **bit de poids faible** est le bit de droite. C'est le seul multiplié par une valeur impaire. C'est donc le bit qui donnera la **parité**. Si la valeur se termine par 0, le nombre sera pair, sinon il sera impair
- le **bit de poids fort** est le bit de gauche.
- un octet est un groupe de 8 bits.

III. Conversion Binaire – Décimal : convertir $1101\,1011_2$ en décimal

Rang du bit	7	6	5	4	3	2	1	0
Puissance de 2	$2^7 = 128$	$2^6 = 64$	$2^5 = 32$	$2^4 = 16$	$2^3 = 8$	$2^2 = 4$	$2^1 = 2$	$2^0 = 1$
Bit	1	1	0	1	1	0	1	1
Valeur	128	64	0	16	8	0	2	1

Ainsi $1101\,1011_2 = 128 + 64 + 16 + 8 + 2 + 1 = 219_{10}$

Exercice 3 : Convertir les valeurs suivantes en décimal : $1001\,0100_2$, 1010_2, $1101\,1101_2$

IV. Conversion Décimal – Binaire

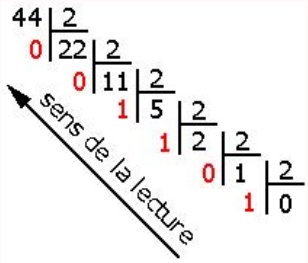
Méthode 1 : Par la méthode des divisions successives

Par exemple pour convertir 44_{10} en binaire :

- On effectue des divisions par 2 jusqu'à ce que le quotient soit nul.
- On lit ensuite les restes de la droite vers la gauche.

On a donc :

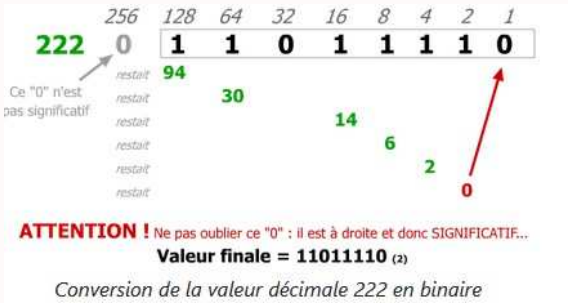
$$44_{10} = 0010\,1100_2$$



Méthode 2 : Par la méthode des soustractions successives

Lorsqu'on cherche à convertir 222_{10}, on remarque que la plus grande puissance de 2 inférieure à 222 est $2^7 = 128$. Nous aurons donc besoin de 8 bits.

- $222 \geqslant 128$. On peut donc décomposer $222 = 2^7 + 94$
- $94 \geqslant 64$. On peut donc décomposer $94 = 2^6 + 30$
- $30 < 36$
- $30 \geqslant 16$. On peut donc décomposer $36 = 2^4 + 14$
- $14 \geqslant 8$. On peut donc décomposer $14 = 2^3 + 6$
- $6 \geqslant 4$. On peut donc décomposer $6 = 2^2 + 2$
- $2 \geqslant 2$. On peut donc décomposer $2 = 2^1 + 0$
- $0 < 1$



Exercice 4 : En utilisant les deux méthodes, convertir 234 **et** 86 **sur 8 bits**

V. Opérations binaires

Propriété 1 : Addition en binaire

L'addition se pratique de la même façon que dans le calcul décimal usuel, et repose sur la table d'addition binaire.

+	0	1
0	00	01
1	01	10

Exercice 5

Poser et effectuer une addition en binaire : $1\,0101_2 + 1110_2$

Propriété 2 : Multiplication en binaire

La multiplication se pratique de la même façon que dans le calcul décimal usuel, et repose sur la table de multiplication suivante :

X	0	1
0	00	00
1	00	01

Exercice 6 : Poser et effectuer une multiplication en binaire : $1\,0101_2 \times 100_2$

Propriété 3 : A retenir

- **Multiplier** une valeur décimale par 2 revient à décaler tous les bits d'un cran vers la gauche et à ajouter un 0 sur le bit de poids faible.
- **Diviser** une valeur décimale par 2 revient à décaler tous les bits d'un cran vers la droite. Le bit de poids faible est perdu. Cela revient à récupérer le quotient de la division par 2.
- **Multiplier** une valeur décimale par 2^n revient à décaler tous les bits de n crans vers la gauche et à ajouter des 0 sur les bits de poids faible.
- **Diviser** une valeur décimale par 2^n revient à décaler tous les bits de n crans vers la droite. Les bit de poids faible sont perdus. Cela revient à récupérer le quotient de la division par 2^n.

VI. L'Hexadécimal

Propriété 1 : Représentation d'un entier en hexadécimal

Si le binaire est une base comprenant 2 éléments, l'hexadécimal est une base comprenant 16 éléments. Il est donc également possible de coder les valeurs décimales suivantes en hexadécimal.
Puisqu'il nous faut 16 symboles. On utilise les 10 premiers chiffres et on rajouter les symboles $\{A, B, C, D, E, F\}$

Valeur décimale	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Valeur hexa																
Valeur décimale	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
Valeur hexa																

Propriété 2 : Conversion hexadécimale

Les méthodes utilisées sont les memes que précédemment en remplaçant la valeur 2 par 16

Exercice 7 : Convertir en décimal : $A3_{16}$, BCF_{16}, 42_{16}

Exercice 8 : Convertir en hexadécimal : 364_{10}, 23_{10}, 44_{10}

Propriété 3 : Conversion binaire – hexadécimal

Dans le cas de conversion binaire - hexadécimal (et inversement) il n'est pas utile de repasser par le décimal. En effet un symbole hexadécimal peut directement etre codé par 4 bits.
Pour convertir des entiers naturels de la représentation binaire à la représentation hexadécimale : Il suffit de regrouper les bits par quatre et de leur associer leur chiffre hexadécimal.
$01101000_2 = 68_{16}$

Exercice 9 : Convertir en hexadécimal :

- 110110100011_2
- 001111111001_2

Exercice 10 : Convertir en binaire :

- $A3E_{16}$
- $8DC_{16}$

Propriété 4 : Conversion en base b

Dans le même ordre d'idée il est donc possible de faire des conversion depuis n'importe quelle base b.

Pour tout entier naturel b non nul appelé base, tout nombre entier naturel n peut se décomposer sous la forme :

$$n = \sum_{k=0}^{N} a_k b^k = a_N b^N + \ldots + a_1 b^1 + a_0 b^0$$

L'écriture en base b de n sera alors $\overline{a_N a_{N-1} \ldots a_1 a_0}$

Méthode 1 : Conversion en python

Code Python

```python
a_bin = 0b1101     # le 0b veut dire que la valeur est écrite en binaire
a_dec = int(a_bin)     # conversion en décimal - valeur entière égale à 13
a_hex = hex(a_dec)     # conversion en hexadécimal - valeur égale à 0xd

b_dec = 44
b_bin = bin(b_dec)     # conversion en binaire - valeur égale à 0b101100
b_hex = hex(b_dec)     # conversion en hexadécimal - valeur égale à 0x2c

# de manière générale, pour convertir un entier n en base b, on peut utiliser:
a_dec = int('1101', 2)      # conversion binaire en décimal égale à 13
a_hex = hex(int('1101', 2))     # conversion binaire en hexadécimal égale à 0xd
a_bin = bin(int('A3', 16))     # conversion hexadécimal en binaire égale à 0b10100011
```

RD03 REPRÉSENTATION DES ENTIERS RELATIFS

Approche intuitive – codage du signe par le bit de poids fort

• le signe + serait codé par un 0 • le signe - serait codé par un 1

Ainsi si $6_{10} = 00000110$ alors $-6_{10} = 10000110$. Plusieurs problèmes se posent à nous :

1. Effectuer l'opération binaire de $-6+6$, qu'obtenez vous en décimal ? .....
2. La valeur 0 aurait donc deux écritures : 00000000 et 10000000

Définition 1 : Le complément à 2 – C2

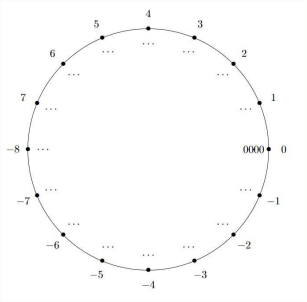
L'idée est de trouver le nombre a, opposé de b tel que $a+b=0$.

Exercice 1 : Déterminer le codage binaire sur 4 bits de -5

Propriété 1 : Codage binaire des entiers relatifs

Complétez le diagramme ci-contre avec tous les codages binaires des entiers relatifs entre -8 et 7 sur 4 bits

- le bit de poids fort des nombres négatifs est 1
- le bit de poids fort des nombres positifs est 0
- il y a autant de positifs que de négatifs (0 est positif)



Propriété 2 : Codage en complément à 2

- Avec n bits en représentation C2 il est possible de coder 2^n valeurs entre -2^{n-1} et $2^{n-1}-1$
- si x est compris entre 0 et $2^{n-1}-1$, le codage binaire de x est celui de l'entier naturel x sur n bits.
- si x n'est pas compris entre -2^{n-1} et $2^{n-1}-1$, alors le codage en C2 sur n bits est impossible.
- Les nombres négatifs commenceront toujours par le bit `1` et les nombres positifs par le bit `0`

Propriété 3 : Complément à 2 – Première méthode

1. Coder la valeur absolue du nombre en binaire
2. Inverser les bits de la représentation du nombre en base 2. (complément à 1)
3. Additionner 1 au résultat précédent.

Exercice 2 : A l'aide de cette méthode, convertir la valeur -5, -7 et -15 sur 4 bits

Propriété 4 : Complément à 2 – Deuxième méthode

1. Coder la valeur absolue du nombre en binaire
2. Recopier les valeurs des bits de droite à gauche jusqu'au premier `1` inclus.
3. Inverser les bits restant jusqu'au dernier (bit de gauche)

Exercice 3 : A l'aide de cette méthodes, convertir la valeur -12, -28 et -112 sur 8 bits

RD04 REPRÉSENTATION DES FLOTTANTS

Pour mémoire, les nombres flottants sont ce que nous appelons en mathématiques les nombres réels ($\mathbb{R}$). A ceci près qu'en Python, on ne peut pas représenter tous les réels mais uniquement les nombres qui auront un nombre fini de chiffres après la virgule. Dans les autres cas, nous devrons faire des approximations.

I. Les nombres dyadiques

Définition 1 : les nombres décimaux

Un nombre décimal est un nombre s'écrvant sous la forme $\frac{x}{10^n}$ où x est un entier relatif et n un entier naturel. Visuellement, un nombre décimal possède un nombre fini de chiffres après la virgule.

Nombre	4	-3	0.25	1/3	1/4	π	10/3
Décimal?	OUI	OUI	OUI	NON	OUI	NON	NON

Définition 2 : les nombres dyadiques

Par analogie, on appelle nombres dyadiques les nombres s'écrivant sous la forme $\frac{x}{2^n}$.

Exercice 1 : Dire si les nombres suivants sont dyadiques

Nombre	13/4	1/5	2.5	25	10/3	π	3.125
Dyadique?							

Définition 3 : Développement dyadique d'un nombre

On appelle développement dyadique d'un nombre l'écriture binaire de ce nombre.

Propriété 1 : Écriture binaire d'un nombre dyadique

Pour obtenir le développement dyadique d'un nombre qui peut s'écrire sous la forme $\frac{x}{2^n}$, on prend le nombre binaire correspondant à x et on insère une virgule avant le n-ième bit en partant de la fin.

Exemple : Écrire le développement dyadique de 2.5

$2.5 = \frac{5}{2^1}$. Le nombre binaire correspondant à 5 est 101_2.
En insérant une virgule avant le premier bit en partant de la fin, on obtient 10.1_2.

Exercice 2 : Trouver les représentations binaires des nombres dyadiques suivants : $\frac{17}{4}$ et $\frac{54}{64}$

II. Conversion d'un nombre décimal en base 2

Méthode 1

1. Conversion de la partie entière du nombre décimal
2. Multiplication de la partie décimale par 2
3. Récupération de la partie entière
4. Si la partie décimale est nulle on arrête sinon retour au 2.
5. Assemblage de la partie entière et des bits récupérés pour former l'écriture binaire

Exemple : Conversion du nombre 5.1875 **en binaire.**

1. Conversion de la partie entière : $5_{10} = 101_2$
2. Multiplications successives par 2 de la partie décimale jusqu'à ce qu'elle soit nulle.

$$\begin{aligned} 0.1875 \times 2 = \quad & 0.375 \quad && = \underline{0} + 0.375 \\ 0.375 \times 2 = \quad & 0.75 \quad && = \underline{0} + 0.75 \\ 0.75 \times 2 = \quad & 1.5 \quad && = \underline{1} + 0.5 \\ 0.5 \times 2 = \quad & 1 \quad && = \underline{1} + 0.0 \end{aligned}$$

On assemble ensuite la partie entière et la partie décimale, ainsi : $5.1875 = 101{,}0011_2$

Exercice 3 : Convertir les nombres suivants :

9.6875 2.625

Remarque

Il existe des nombres décimaux non dyadiques ! Ainsi si on applique la méthode ci-dessus, alors le développement ne s'arrêtera pas. C'est pour cela qu'il faudra définir une précision souhaitée. Et cela explique aussi certains comportements étranges au premier abord de Python.

```
>>> print(0.1 + 0.1)
0.2
>>> print(0.1 + 0.1 + 0.1)
0.30000000000000004
```

- En Python, les flottants sont codés sur 64 bits
- Il faut éviter de tester l'égalité de deux flottants.

Exercice 4 : Expliquer par le calcul le résultat de l'addition $1.1 + 0.1$

III. De l'écriture binaire à la notation décimale

Exemple : Convertir $100{,}0101_2$

1. Convertir la partie entière : $100_2 = 4_{10}$
2. Convertir la partie décimale. On utilise un tableau des puissances de 2 négative.

Rang du bit	-1	-2	-3	-4	-5	-6
Puissance de 2	$2^{-1} = 0.5$	$2^{-2} = 0.25$	$2^{-3} = 0.125$	$2^{-4} = 0.0625$	$2^{-5} = 0.03125$	$2^{-6} = 0.015625$
Bit	0	1	0	1	0	0
Valeur	0	0.25	0	0.0625	0	0

Il suffit ensuite d'additionner toutes les valeurs calculées : $100{,}0101_2 = 4 + 0.25 + 0.0625 = 4.3125_{10}$

Exercice 5 : Trouver la représentation décimale de 101.001

1. Convertir la partie entière : .....
2. Convertir la partie décimale. On utilise un tableau des puissances de 2 négative.

Rang du bit	-1	-2	-3	-4	-5	-6
Puissance de 2	$2^{-1} = 0.5$	$2^{-2} = 0.25$	$2^{-3} = 0.125$	$2^{-4} = 0.0625$	$2^{-5} = 0.03125$	$2^{-6} = 0.015625$
Bit						
Valeur						

Exercice 6 : Convertir en décimal les nombres suivants :

1101,100101 1000,11101

IV. Notation scientifique

En base dix, il est possible d'écrire les très grands nombres et les très petits nombres grâce aux "puissances de dix" (exemples $6{,}02 \times 10^{23}$ ou $6{,}67 \times 10^{-11}$).

Il est possible de faire exactement la même chose avec une représentation binaire, puisque nous sommes en base 2, nous utiliserons des "puissances de deux" à la place des "puissances dix" (exemple $101{,}1101.2^{10}$).

Exemple : Pour passer d'une écriture sans "puissance de deux" à une écriture avec "puissance de deux", il suffit décaler la virgule : $1101{,}1001 = 1{,}1011001.2^{11}$.

Pour passer de 1101,1001 à 1,1011001 nous avons décalé la virgule de 3 rangs vers la gauche d'où le 2^{11}. (attention de ne pas oublier que nous travaillons en base 2 le " 11 " correspond bien à un décalage de 3 rangs de la virgule, car $11_2 = 3_{10}$.

Exemple : Si l'on désire décaler la virgule vers la gauche, il va être nécessaire d'utiliser des "puissances de deux négatives". $0{,}0110_2 = 1{,}10.2^{-10}$. Nous décalons la virgule de 2 rangs vers la droite, d'où le " -10 ".

Exercice 7 : Trouver l'écriture binaire scientifique de :

1. $1011{,}0111101_2$ .....
2. 0.0745_{10} .....

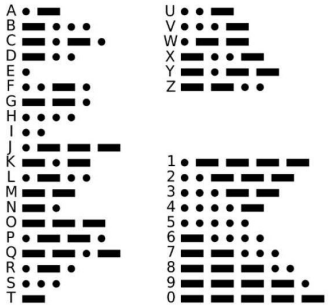
RD05 REPRÉSENTATION DU TEXTE

Quelle que soit la nature de l'information traitée par un ordinateur (image, son, texte, vidéo), elle est toujours représentée sous la forme d'un ensemble de nombres binaires.

I. Le codage Morse

Un des premiers systèmes de codage est le code Morse :

- Il a été développé par Samuel Morse et ses collaborateurs (1837)
- Il a servi pour la transmission de signaux par télégraphe
- Le système de codage est fait avec des séquences signal court/long



Le code Morse est au départ principalement utilisé par les militaires pour transmettre des informations, souvent chiffrées.

Le signal peut être transporté via un signal radio, des signaux électriques à travers un câble télégraphique ou des signaux visuels (lumière).

L'idée est de coder les caractères fréquents avec des séquences simples et les caractères rares par des séquences compliquées.

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[ENG OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	|
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

Exercice 1 : Coder en ASCII le mot suivant : Bonjour

II. Codage ISO 8859-1

La norme ASCII convient bien à la langue anglaise, mais pose des problèmes dans d'autres langues, par exemple le français. En effet l'ASCII ne prévoit pas d'encoder les lettres accentuées.

La norme ISO-8859-1 reprend les mêmes principes que l'ASCII, mais les nombres binaires associés à chaque caractère sont codés sur 8 bits, ce qui permet d'encoder jusqu'à 256 caractères.

D'autres normes ont donc dû voir le jour, par exemple la norme "GB2312" pour le chinois simplifié ou encore la norme "JIS X 0208" pour le japonais.

ISO/CEI 8859-1

	x0	x1	x2	x3	x4	x5	x6	x7	x8	x9	xA	xB	xC	xD	xE	xF
0x	positions inutilisées															
1x																
2x	SP	!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3x	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4x	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5x	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6x	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7x	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	
8x	positions inutilisées															
9x																
Ax	NBSP	¡	¢	£	¤	¥	¦	§	¨	©	ª	«	¬	-	®	¯
Bx	°	±	²	³	´	µ	¶	·	¸	¹	º	»	¼	½	¾	¿
Cx	À	Á	Â	Ã	Ä	Å	Æ	Ç	È	É	Ê	Ë	Ì	Í	Î	Ï
Dx	Ð	Ñ	Ò	Ó	Ô	Õ	Ö	×	Ø	Ù	Ú	Û	Ü	Ý	Þ	ß
Ex	à	á	â	ã	ä	å	æ	ç	è	é	ê	ë	ì	í	î	ï
Fx	ð	ñ	ò	ó	ô	õ	ö	÷	ø	ù	ú	û	ü	ý	þ	ÿ

Exercice 2 : Décoder le texte suivant : 4C 61 20 43 69 67 61 6C 65 20 65 74 20 62 62 20 46 6F 75 72 6D 69 2E 20

III. Codage Unicode

Unicode a pour ambition de rassembler tous les caractères existant afin qu'une personne utilisant Unicode puisse, sans changer la configuration de son traitement de texte, à la fois lire des textes en français ou en japonais.

Unicode est uniquement une table qui regroupe tous les caractères existant au monde, il ne s'occupe pas de la façon dont les caractères sont codés dans la machine. Unicode accepte plusieurs systèmes de codage : UTF-8, UTF-16, UTF-32. Le plus utilisé, notamment sur le Web, est UTF-8.

Pour encoder les caractères Unicode, UTF-8 utilise un nombre variable d'octets : les caractères "classiques" (les plus couramment utilisés) sont codés sur un octet, alors que des caractères "moins classiques" sont codés sur un nombre d'octets plus important (jusqu'à 4 octets).

Un des avantages d'UTF-8 est qu'il est totalement compatible avec la norme ASCII : Les caractères Unicode codés avec UTF-8 ont exactement le même code que les mêmes caractères en ASCII.

On retrouve ici le codage 8 bits :

`http://sebastienguillon.com/test/jeux-de-caracteres/windows-ascii-fr.html`

Propriété 1 : Codage UTF-8

- On transforme le code point en binaire
- On code entre 1 et 4 octets selon la table suivante :

Number of bytes	Bits for code point	First code point	Last code point	Byte 1	Byte 2	Byte 3	Byte 4
1	7	U+0000	U+007F	0xxxxxxx			
2	11	U+0080	U+07FF	110xxxxx	10xxxxxx		
3	16	U+0800	U+FFFF	1110xxxx	10xxxxxx	10xxxxxx	
4	21	U+10000	U+10FFFF[12]	11110xxx	10xxxxxx	10xxxxxx	10xxxxxx

Les "xxx" sont les chiffres de la représentation du code point en base 2.

Exercice 3 : Coder en UTF-8 la lettre é (U+00E9), le symbole ∅, → et ↑

Propriété 2 : Unicode et Python

En python il est possible de comparer des chaines de caractère directement

Code Python

```python
>>> 'abc' < 'abd'
True
>>> 'ab' < 'abc'
True
```

En python il existe des fonctions capables de transformer les caractère en nombre et les nombres en caractère.

Code Python

```python
>>> chr(65)      # valeur décimale -> caractère
'A'
>>> ord('A')     # caractère -> valeur décimale
65
```